Inventory AI

Design of a Hybrid LLM-based Query System for University Inventory Management

Óbudai Egyetem
Mesterséges Intelligencia specializáció

Projektmunka II.

Készítette:
Vincze Balázs

Dátum:
2026. március 9.

Tartalomjegyzék

1. A rendszer célja
2. Példa működés
3. Fő architektúra áttekintése
4. Query Routing rendszer
5. Technológiai stack
6. SQL Engine működése
7. RAG Engine működése
8. Hybrid válasz generálás
9. Adatbázis séma
10. Miért Hybrid rendszer?
11. Az első hét eredménye

Architektúra tervezés: Hybrid LLM Query System

1. A rendszer célja

Az **Inventory AI** projekt célja egy olyan mesterséges intelligencia alapú chatbot rendszer megtervezése, amely lehetővé teszi az egyetemi leltáradatok és a kapcsolódó szakmai dokumentációk természetes nyelvű lekérdezését. A rendszer elsődleges feladata, hogy a felhasználó számára egyszerű, intuitív és emberközeli módon biztosítson hozzáférést olyan információkhoz, amelyek hagyományos megközelítésben különböző adatforrásokban, eltérő formátumban és eltérő keresési módszerekkel érhetők el.

A projekt alapfeltevése, hogy az egyetemi vagy laboratóriumi környezetben használt eszközökhöz két, egymást kiegészítő információtípus tartozik. Egyrészt rendelkezésre állnak a **strukturált inventory adatok**, például az adott eszköz neve, kategóriája, darabszáma, helye, állapota vagy karbantartási nyilvántartása. Másrészt jelen vannak a **nem strukturált szakmai dokumentumok**, például használati útmutatók, műszaki leírások, kalibrálási dokumentációk vagy karbantartási előírások. Egy hatékony intelligens rendszernek ezért nem elegendő kizárólag adatbázis-lekérdezéseket végrehajtania, és nem elegendő pusztán dokumentumokban keresnie sem; a valós felhasználói igények kielégítéséhez a két megközelítés együttes alkalmazására van szükség.

A rendszer célja tehát egy **hybrid LLM alapú architektúra** kialakítása, amely képes több információforrás intelligens összekapcsolására. A rendszernek automatikusan fel kell ismernie, hogy a felhasználó kérdése milyen típusú választ igényel, majd ennek megfelelően ki kell választania a megfelelő feldolgozási útvonalat. Ennek megfelelően a megoldásnak három alapvető működési módot kell támogatnia.

Az első működési mód a **strukturált adatbázis-lekérdezés**, amely olyan kérdések esetén szükséges, amikor a felhasználó konkrét inventory adatokra kíváncsi. Ilyen lehet például egy eszköz helyének meghatározása, a rendelkezésre álló darabszám lekérdezése, egy adott kategóriába tartozó eszközök listázása vagy a készletállapot vizsgálata. Ezekben az esetekben a rendszernek természetes nyelvű kérdésből SQL lekérdezést kell előállítania, majd az eredményt a felhasználó számára érthető formában kell visszaadnia.

A második működési mód a **dokumentumalapú keresés**, amely akkor szükséges, amikor a kérdés nem strukturált szakmai tudást igényel. Ebbe a kategóriába tartoznak például a használati útmutatókra, működési leírásokra, beállítási folyamatokra, kalibrálási lépésekre vagy karbantartási eljárásokra vonatkozó kérdések. Ezekben az esetekben a rendszernek egy Retrieval-Augmented Generation (RAG) alapú megoldást kell alkalmaznia, amely a releváns dokumentumrészletek visszakeresése után nyelvi modell segítségével állít elő természetes nyelvű választ.

A harmadik működési mód a **hybrid válasz-előállítás**, amely a rendszer legfontosabb és legösszetettebb funkciója. Ennek során a felhasználói kérdés egyszerre igényel strukturált adatokat és dokumentációs ismereteket. Például egy olyan kérdés, mint „Hány 3D nyomtató áll rendelkezésre, és hogyan kell őket karbantartani?” egyszerre követel pontos adatbázis-lekérdezést és szakmai dokumentációs tudást. A rendszernek ilyen esetben mindkét forrást fel

kell használnia, majd az így nyert eredményeket egységes, jól értelmezhető válaszban kell összefoglalnia.

A projekt további célja, hogy a rendszer működése **moduláris, bővíthető és biztonságos** legyen. A moduláris felépítés azért fontos, mert így az egyes komponensek — például a backend, a query router, a SQL engine, a RAG engine vagy a válaszgeneráló modul — egymástól elkülönítve fejleszthetők és tesztelhetők. A bővíthetőség lehetővé teszi, hogy a rendszer később új adatforrásokkal, új dokumentumtípusokkal vagy fejlettebb modellekkel egészülhessen ki. A biztonság pedig különösen fontos az adatbázis-kezelés területén, mivel a természetes nyelvből generált SQL lekérdezések esetében garantálni kell, hogy a rendszer kizárólag olvasási célú, ellenőrzött műveleteket hajtson végre.

A rendszer tervezése során kiemelt szempont az is, hogy a felhasználó számára a teljes folyamat **egyetlen egységes intelligens szolgáltatásként** jelenjen meg. A végfelhasználónak ne kelljen tudnia, hogy a válasz adatbázisból, dokumentumokból vagy több forrás kombinációjából származik; elegendő legyen természetes nyelven megfogalmaznia a kérdését, a rendszer pedig a háttérben önállóan elvégezze a szükséges döntéseket és feldolgozási lépéseket.

Összességében az Inventory AI rendszer célja egy olyan korszerű, mesterséges intelligenciára épülő lekérdező platform létrehozása, amely képes áthidalni a strukturált adatkezelés és a dokumentumalapú tudáskeresés közötti különbséget. A rendszer ezzel nemcsak az inventory adatok elérését teszi hatékonyabbá, hanem jelentősen javítja a szakmai dokumentációk használhatóságát is, ezáltal támogatva az egyetemi, laboratóriumi és műszaki környezetben zajló napi munkát.

---

2. Példa működés

a. SQL kérdés

Felhasználó:

„Hol található a projektor?”

A rendszer felismeri, hogy ez **strukturált adat**, ezért:

---

b. Dokumentáció kérdés

Felhasználó:

„Hogyan kell kalibrálni a 3D nyomtatót?”

Ez **nem adatbázis kérdés**, ezért:

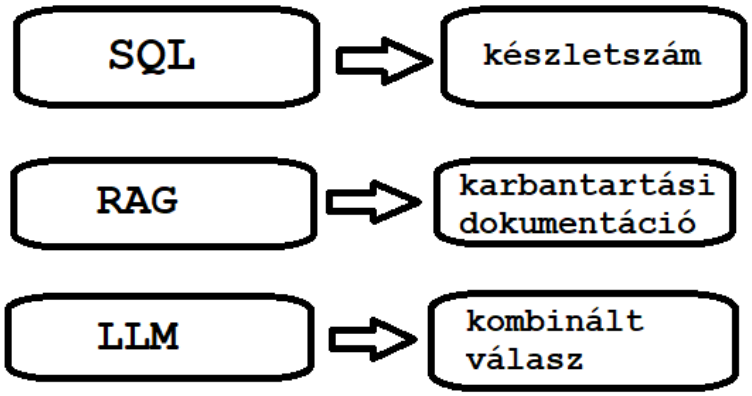
---

c. Hybrid kérdés

Felhasználó:

„Mennyi 3D nyomtató van és hogyan kell karbantartani?”

A rendszer két forrást használ:



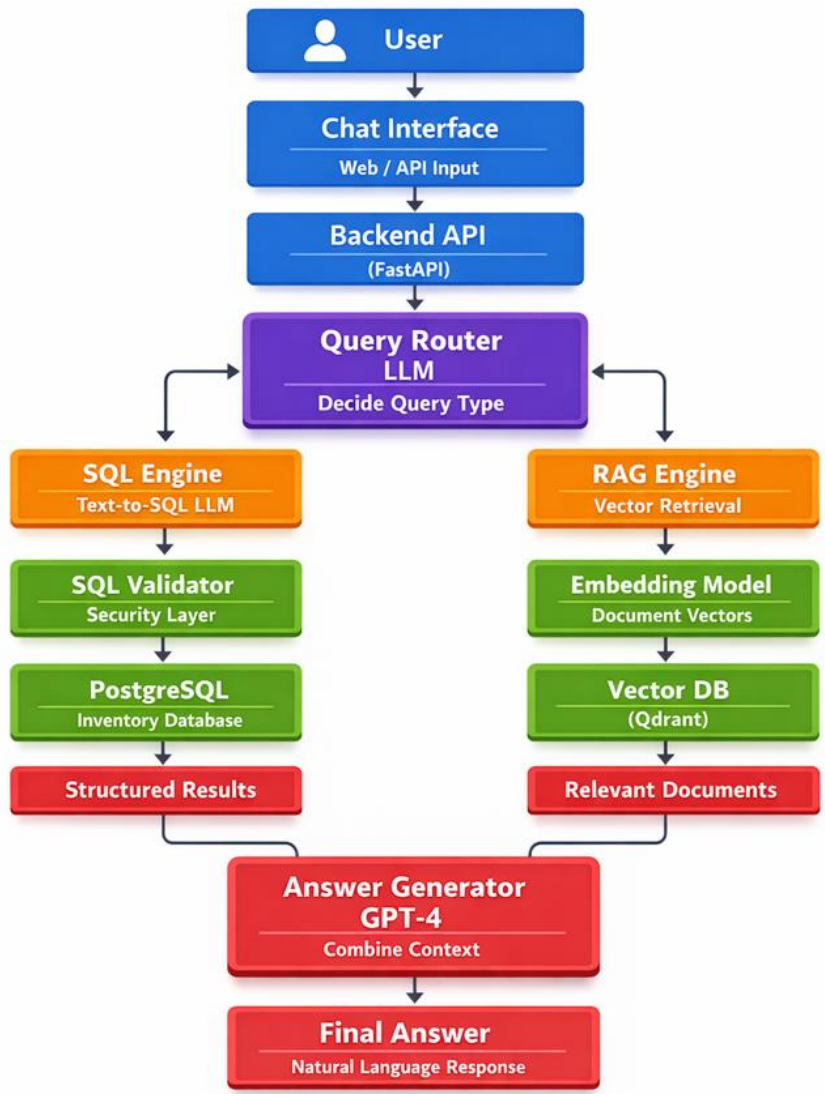
---

3. Fő architektúra áttekintése

Ez a fejezet a rendszer teljes architektúrájának magas szintű áttekintését mutatja be. Az alábbi ábra szemlélteti az Inventory AI rendszer fő komponenseit és azok egymáshoz való kapcsolatát.

Az architektúra bemutatja a felhasználói kérdés feldolgozásának teljes folyamatát, a kérdés beérkezésétől kezdve egészen a végső válasz generálásáig. A diagram tartalmazza a rendszer legfontosabb elemeit, például a backend API-t, a Query Router modult, a SQL és RAG feldolgozó komponenseket, valamint a válaszgeneráló réteget.

Az ábra célja, hogy áttekintést adjon a rendszer működéséről és az egyes modulok szerepéről. A következő fejezetekben az architektúra egyes komponensei részletesen bemutatásra kerülnek, beleértve a Query Routing mechanizmust, a SQL Engine működését, a RAG alapú dokumentumkeresést, valamint a hybrid válaszgenerálási folyamatot.



4. Query Routing rendszer

A rendszer egyik legfontosabb komponense a **Query Router**, amely a felhasználói kérdések feldolgozásának első lépése. Miután a felhasználó természetes nyelvű kérdést tesz fel a chat interfészen keresztül, a kérdés a backendhez kerül, majd onnan a Query Router modulhoz továbbítódik.

A Query Router feladata annak eldöntése, hogy a kérdés megválaszolásához melyik adatforrást kell használni. A rendszer két fő adatforrással dolgozik:

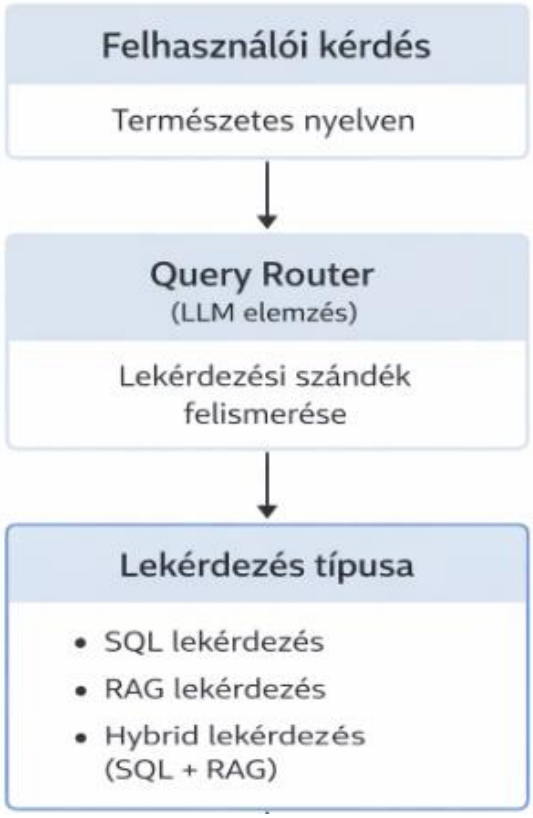
- strukturált adatok az **inventory adatbázisban**
- nem strukturált információk **dokumentációkban**

A router egy **LLM alapú döntési mechanizmust** használ a kérdés elemzésére. A modell a kérdés jelentését vizsgálja, majd eldönti, hogy melyik feldolgozási útvonal a legmegfelelőbb.

A router három különböző kategóriába sorolhatja a kérdéseket:

- SQL lekérdezés
- RAG dokumentumkeresés
- Hybrid feldolgozás

A feldolgozási folyamat:



4.1 SQL routing

A router SQL kategóriába sorolja azokat a kérdéseket, amelyek **strukturált adatokat igényelnek** az inventory adatbázisból.

Ezek általában olyan kérdések, amelyek:

- eszközök **helyszínére** kérdeznek rá
- eszközök **mennyiségére** vonatkoznak
- konkrét **inventory itemet** keresnek
- készletinformációkat igényelnek

Példák

- „Hol van a projektor?”
- „Hány laptop van raktáron?”
- „Melyik teremben található a mikroszkóp?”
- „Hány darab oszcilloszkóp van a laborban?”

Ebben az esetben a rendszer a kérdést a **SQL Engine** komponenshez továbbítja.

4.2 RAG routing

A RAG kategóriába tartoznak azok a kérdések, amelyek **dokumentációs információkat** igényelnek.

Ezek jellemzően:

- használati útmutatók
- működési leírások
- kalibrálási folyamatok
- karbantartási instrukciók

Példák

- „Hogyan kell kalibrálni a mikroszkópot?”
- „Hogyan működik a spektrométer?”
- „Hogyan kell használni a 3D nyomtatót?”

Ebben az esetben a rendszer a **RAG Engine** komponenshez irányítja a kérdést.

4.3 Hybrid routing

Hybrid feldolgozás akkor szükséges, amikor a kérdés **egyszerre igényel strukturált adatot és dokumentációs információt**.

Ilyenkor a rendszer két külön feldolgozási folyamatot indít:

- SQL lekérdezést
- dokumentumkeresést

Példák

- „Hány oszcilloszkóp van és hogyan kell használni?”
- „Mennyi projektor van és hogyan kell karbantartani?”
- „Hány 3D nyomtató van és hogyan kell kalibrálni?”

A feldolgozás során az SQL és RAG eredmények összevonásra kerülnek.

---

5. Technológiai stack

A rendszer implementációja több különböző technológiai komponensre épül, amelyek együtt biztosítják a természetes nyelvű kérdések feldolgozását.

5.1 LLM modell

A rendszer központi intelligens komponense egy **nagy nyelvi modell (LLM)**.

A projektben a **GPT-4 modell** kerül alkalmazásra.

Az LLM több feladatot lát el:

- kérdésértelmezés
- query routing
- SQL generálás
- válaszgenerálás

A modell képes természetes nyelvű kérdésekből strukturált lekérdezéseket generálni, valamint dokumentumok alapján válaszokat készíteni.

5.2 Backend rendszer

A rendszer backend komponense **Python alapú szerveralkalmazás**.

A backend fő technológiái:

- **FastAPI**
- **LangChain vagy egyéb LLM wrapper**

Backend feladatai

- felhasználói kérdések fogadása
- Query Router meghívása
- SQL és RAG engine koordinálása
- adatforrások elérése
- válasz generálása és visszaküldése

A backend tehát a rendszer **központi vezérlő rétege**.

5.3 Relációs adatbázis

A strukturált adatok tárolására a rendszer **PostgreSQL adatbázist** használ.

Ez tárolja az inventory rendszer adatait, például:

- eszközök neve
- kategóriák
- helyszínek
- készletszám
- státusz
- karbantartási adatok

A PostgreSQL adatbázis biztosítja a strukturált lekérdezések hatékony végrehajtását.

5.4 Vector adatbázis

A dokumentumkereséshez a rendszer **vector adatbázist** használ.

A projektben a választott megoldás:

Qdrant

A vector adatbázis tárolja a dokumentumok embedding reprezentációját.

A rendszerben tárolt dokumentumtípusok például:

- PDF dokumentációk
- használati útmutatók
- karbantartási leírások
- technikai dokumentumok

A vector adatbázis lehetővé teszi a **szemantikai keresést**.

---

6. SQL Engine működése

A SQL Engine feladata a természetes nyelvű kérdések **SQL lekérdezéssé alakítása** és az adatbázis lekérdezése.

A folyamat lépései:

6.1 SQL generálás

Az LLM a következő információk alapján generál SQL lekérdezést:

- adatbázis séma
- táblák és mezők
- lekérdezési szabályok
- felhasználói kérdés

Példa

Felhasználói kérdés:

„Hol található a projektor?”

A modell a következő SQL lekérdezést generálhatja:

```
SELECT location
FROM inventory_items
WHERE name ILIKE '%projektor%'
LIMIT 20;
```

6.2 SQL validáció

A generált SQL lekérdezés egy **biztonsági ellenőrzési rétegen** halad át.

A SQL Validator ellenőrzi:

- a lekérdezés típusát
- az engedélyezett SQL kulcsszavakat
- a táblahivatkozásokat

A rendszer kizárólag **SELECT lekérdezéseket** engedélyez.

Tiltott parancsok:

- DROP
- DELETE
- UPDATE
- INSERT
- ALTER

Ez megakadályozza az adatbázis módosítását vagy sérülését.

6.3 Lekérdezés végrehajtása

A validált SQL lekérdezés a **PostgreSQL adatbázison** kerül végrehajtásra.

Az eredmény strukturált formában érkezik vissza.

Példa:

```
location: Labor 2
```

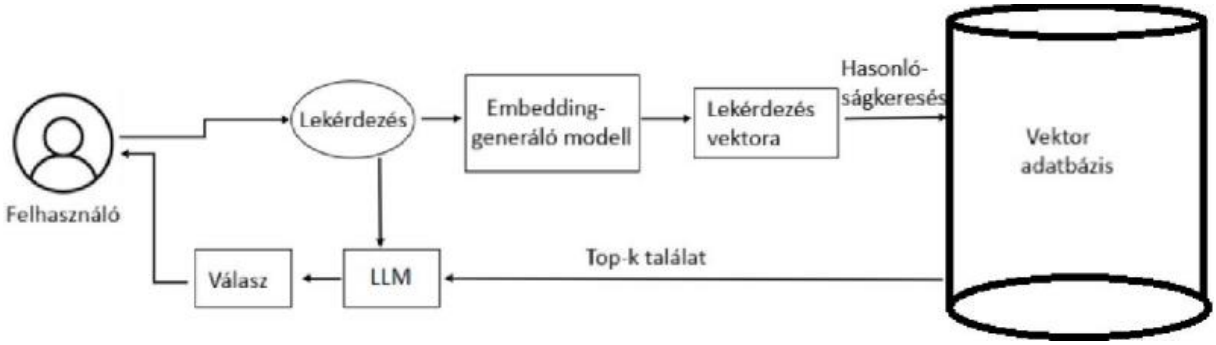
A backend ezt természetes nyelvű válasszá alakítja.

---

7. RAG Engine működése

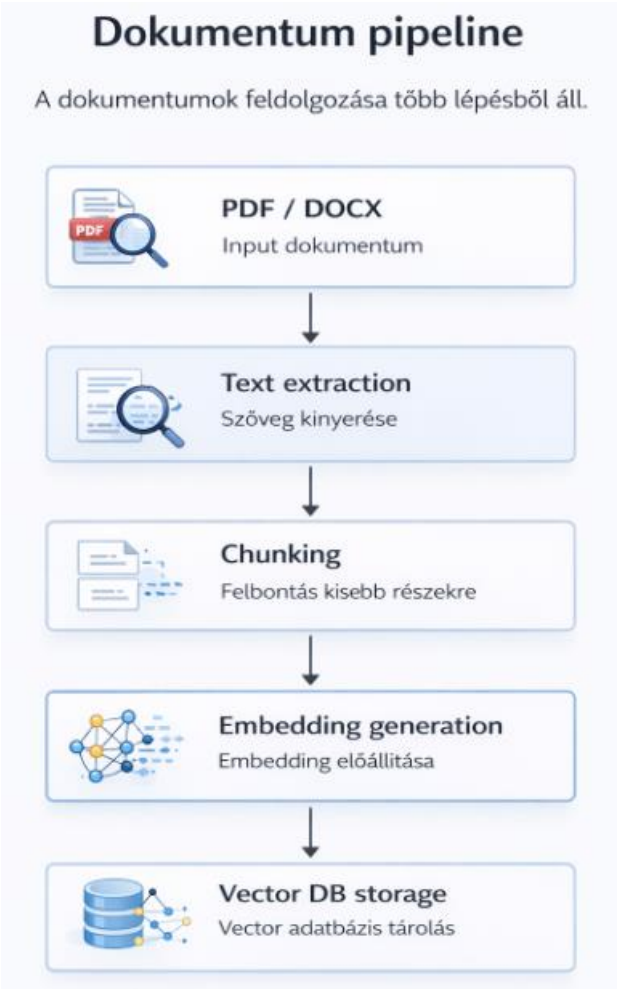
A RAG Engine a dokumentumokban található információk keresésére szolgál.

A RAG (Retrieval Augmented Generation) működése:



7.1 Dokumentum pipeline

A dokumentumok feldolgozása több lépésből áll.



7.2 Embedding generálás

A RAG rendszer működésének egyik alaplépése az **embedding generálás**. Ennek során a dokumentumok szöveges tartalma numerikus vektorrá alakul, amely a szöveg jelentésének matematikai reprezentációját adja.

Az embedding modell lehetővé teszi, hogy a rendszer a szövegek közötti **szemantikai hasonlóságot** vizsgálja. A dokumentumokat kisebb szakaszokra (chunkokra) bontjuk, majd ezekhez külön embedding vektorok készülnek. Ezek a vektorok kerülnek eltárolásra a vector adatbázisban.

7.3 Dokumentumkeresés

Amikor a felhasználó kérdést tesz fel, a kérdés is embedding reprezentációvá alakul. A rendszer ezután a kérdés vektorát összehasonlítja a vector adatbázisban tárolt dokumentumvektorokkal, és kiválasztja a legnagyobb hasonlóságot mutató dokumentumrészleteket.

A folyamat lépései:

1. a felhasználói kérdés embedding reprezentációt kap
2. a rendszer hasonlósági keresést végez a vector adatbázisban
3. a legrelevánsabb dokumentumrészleteket visszaadja

7.4 Példa

Felhasználó:

„Hogyan kell kalibrálni a spektrométert?”

A rendszer:

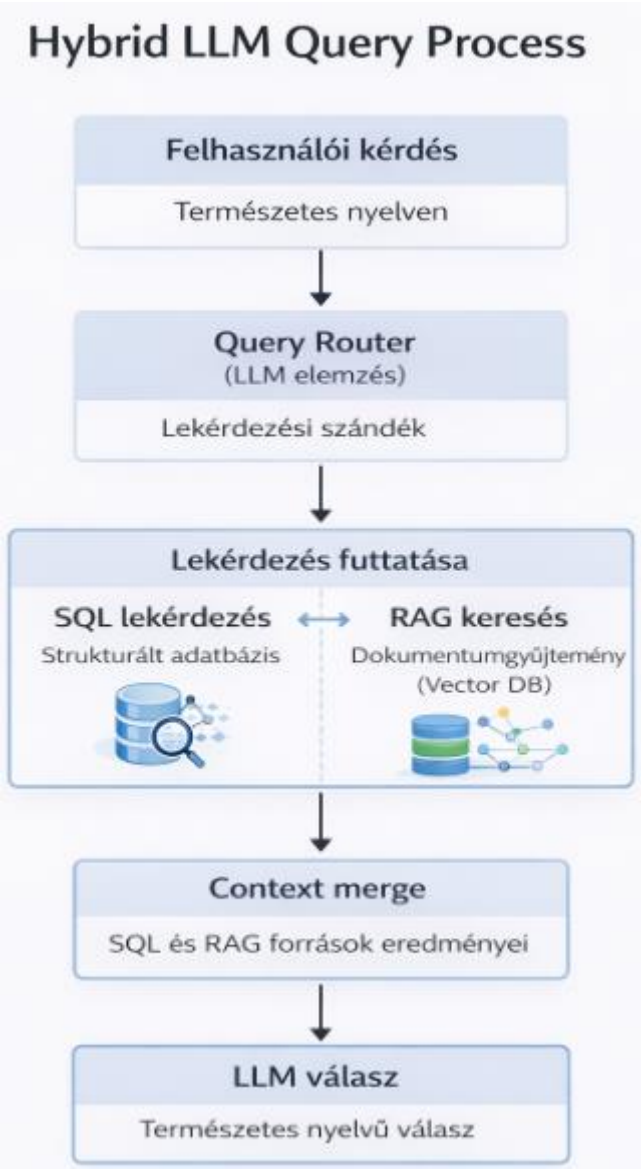
1. embeddinget készít a kérdésből
2. releváns dokumentumrészleteket keres a vector adatbázisban
3. a dokumentumrészleteket átadja az LLM-nek
4. az LLM ezek alapján generál természetes nyelvű választ.

---

8. Hybrid válasz generálás

Hybrid kérdések esetén a rendszer **két adatforrást kombinál**.

A feldolgozási folyamat:



<u>Példa</u>

Felhasználó:

„Hány spektrométer van és hogyan kell kalibrálni?”

SQL eredmény:

```
5 darab spektrométer
```

Dokumentum eredmény:

`kalibrálási útmutató`

LLM válasz:

„Az adatbázis szerint 5 spektrométer található a laborban.
A kalibrálási folyamat során először ellenőrizni kell a referencia mintát, majd be kell állítani a mérési paramétereket.”

---

9. Adatbázis séma

A rendszer strukturált adatai relációs adatbázisban kerülnek tárolásra.

9.1 Inventory tábla

`inventory_items`

mezők:

- id
- name
- category
- location
- quantity
- status
- description

9.2 Karbantartási log

`maintenance_logs`

mezők:

- id
- item_id
- maintenance_date
- technician
- notes

---

10. Miért Hybrid rendszer?

A hybrid architektúra kombinálja az SQL és RAG rendszerek előnyeit.

A klasszikus **RAG rendszerek** jól működnek dokumentumok keresésében, de nem alkalmasak strukturált adatok pontos lekérdezésére.

Az **SQL rendszerek** viszont kiválóan kezelik a strukturált adatokat, de nem tudnak komplex dokumentációs kérdésekre válaszolni.

A két technológia kombinációja lehetővé teszi, hogy a chatbot:

- inventory adatokat kezeljen
- technikai dokumentációt értelmezzen
- komplex kérdésekre válaszoljon

---

11. Az első hét eredménye

Az első fejlesztési hét során a rendszer alapvető architektúrája került kialakításra.

Az elért eredmények:

- a rendszer architektúrájának megtervezése
- a technológiai stack kiválasztása
- a SQL Engine koncepciójának meghatározása
- a RAG integráció megtervezése
- a Query Routing modell kialakítása

Ez a fázis biztosította az alapot a következő fejlesztési lépésekhez, amelyek során a rendszer tényleges implementációja történik.